



Materiały dydaktyczne

Przedmiot: Programowanie obiektowe/Pracownia programowania obiektowego

Opracował: Mirosław Miciak

Ćwiczenie: C13

Tematy: Podział programu na funkcje, Biblioteki języka programowania, Proste i złożone typy danych, Zmienne, Operacje na zmiennych: wejścia, wyjścia, arytmetyczne logiczne, Operacje na typach złożonych, Operatory arytmetyczne, przypisania, porównania, Operatory logiczne i bitowe, Operatory do obsługi łańcuchów, Instrukcja warunkowa i instrukcja wyboru, Instrukcje pętli, Wykorzystanie wybranej biblioteki matematycznej, Wykorzystanie bibliotek algorytmów, Funkcje, Klasa, obiekt, metoda, pole, Projektowanie prostych klas, Zmienne obiektowe, Zakres widoczności pól klasy, Metody klasy, Konstruktor i destruktor, Konstruktor kopiujący, Metody klasy - zakres widoczności, Prosta aplikacja obiektowa: konstruktor, destruktor, Składniki statyczne klasy, Klasy dziedziczone, Klasy zaprzyjaźnione, Dziedziczenie, Hermetyzacja, Polimorfizm, Funkcje zaprzyjaźnione z klasą, Metody do obsługi elementów statycznych klasy, Klasy bazowe i pochodne, Metody wirtualne, Klasy abstrakcyjne, Szablony klas dla prostych typów liczbowych, Obsługa wyjątków, Instrukcja throw, Obsługa błędów wykonania aplikacji.

Instrukcja throw w programowaniu obiektowym

Instrukcja throw w języku C# jest używana do ręcznego zgłaszania wyjątków. Wyjątki są mechanizmem, który pozwala na obsługę błędów i nieoczekiwanych sytuacji w trakcie wykonywania programu. Dzięki nim możemy kontrolować przepływ programu w przypadku wystąpienia błędów, co zwiększa jego niezawodność i ułatwia debugowanie.

W programowaniu obiektowym, wyjątki są często używane do sygnalizowania problemów związanych z nieprawidłowym stanem obiektu lub nieprawidłowym użyciem metod. Instrukcja throw pozwala na zgłoszenie wyjątku w dowolnym miejscu w kodzie, co umożliwia przerwanie normalnego przepływu programu i przejście do bloku catch, gdzie wyjątek może zostać obsłużony.

Przykład 1. Proste użycie instrukcji throw

```
using System;

class Program
{
    static void Main(string[] args)
    {
        try
        {
            Console.WriteLine("Podaj liczbę:");
            int liczba = int.Parse(Console.ReadLine());

            if (liczba < 0)
            {
                // Ręczne zgłoszenie wyjątku, jeśli liczba jest ujemna
                throw new ArgumentOutOfRangeException("Liczba nie może być ujemna.");
            }

            Console.WriteLine($"Podana liczba to: {liczba}");
        }
    }
}
```

```

        catch (ArgumentOutOfRangeException ex)
        {
            // Obsługa wyjątku
            Console.WriteLine(ex.Message);
        }
        catch (FormatException)
        {
            // Obsługa wyjątku związanego z nieprawidłowym formatem liczby
            Console.WriteLine("Podano nieprawidłowy format liczby.");
        }
        finally
        {
            // Blok finally wykonuje się niezależnie od tego, czy wystąpił wyjątek
            Console.WriteLine("Koniec programu.");
        }
    }
}

```

W powyższym przykładzie użytkownik jest proszony o podanie liczby. Jeśli liczba jest ujemna, zgłaszany jest wyjątek `ArgumentOutOfRangeException` za pomocą instrukcji `throw`. Wyjątek jest przechwytywany w bloku `catch`, gdzie wyświetlany jest odpowiedni komunikat. Blok `finally` wykonuje się niezależnie od tego, czy wystąpił wyjątek, co jest przydatne do zwalniania zasobów.

Przykład 2. Tworzenie własnych wyjątków

```

using System;

// Własna klasa wyjątku
class MojaWyjatkowaSytuacja : Exception
{
    public MojaWyjatkowaSytuacja(string message) : base(message)
    {
    }
}

class Program
{
    static void Main(string[] args)
    {
        try
        {
            Console.WriteLine("Podaj imię:");
            string imie = Console.ReadLine();

            if (imie.Length < 3)
            {
                // Ręczne zgłoszenie własnego wyjątku
                throw new MojaWyjatkowaSytuacja("Imię musi mieć co najmniej 3 znaki.");
            }

            Console.WriteLine($"Witaj, {imie}!");
        }
        catch (MojaWyjatkowaSytuacja ex)
        {
            // Obsługa własnego wyjątku
            Console.WriteLine(ex.Message);
        }
        finally
        {
            Console.WriteLine("Koniec programu.");
        }
    }
}

```

W tym przykładzie tworzymy własną klasę wyjątku `MojaWyjatkowaSytuacja`, która dziedziczy po klasie `Exception`. Jeśli użytkownik poda imię krótsze niż 3 znaki, zgłaszany jest ten wyjątek. Wyjątek jest przechwytywany w bloku `catch`, gdzie wyświetlany jest odpowiedni komunikat.

Instrukcja `throw` jest narzędziem w programowaniu obiektowym, które pozwala na ręczne zgłaszanie wyjątków w przypadku nieprawidłowych sytuacji. Dzięki niej możemy kontrolować przepływ programu i zapewnić, że błędy są odpowiednio obsługiwane. W powyższych przykładach i zadaniach pokazano, jak używać `throw` w praktyce, tworzyć własne wyjątki oraz jak je obsługiwać.

Zadania do samodzielnego rozwiązania

Zadanie 1. Obsługa wyjątków w kalkulatorze

Napisz program kalkulatora, który wykonuje podstawowe operacje matematyczne (dodawanie, odejmowanie, mnożenie, dzielenie). Użyj instrukcji `throw`, aby zgłosić wyjątek, jeśli użytkownik spróbuje podzielić przez zero. Dodaj również obsługę wyjątków związanych z nieprawidłowym formatem danych wejściowych.

Wskazówka:

- Użyj instrukcji `throw` w przypadku dzielenia przez zero.
- Użyj bloków `catch` do obsługi różnych typów wyjątków.

Program prosi użytkownika o podanie dwóch liczb i wybór operacji. W przypadku dzielenia przez zero, zgłaszany jest wyjątek `DivideByZeroException`. Program obsługuje również wyjątki związane z nieprawidłowym formatem liczby (`FormatException`) oraz nieprawidłową operacją (`InvalidOperationException`).

Zadanie 2. Własny wyjątek dla walidacji hasła

Stwórz program, który prosi użytkownika o podanie hasła. Hasło musi spełniać następujące wymagania:

- Musi mieć co najmniej 8 znaków.
- Musi zawierać co najmniej jedną cyfrę.
- Musi zawierać co najmniej jedną wielką literę.

Jeśli hasło nie spełnia tych wymagań, zgłoś własny wyjątek `NieprawidloweHasloException`. Przechwyć ten wyjątek i wyświetl odpowiedni komunikat.

Program sprawdza, czy hasło spełnia wymagania (co najmniej 8 znaków, jedna cyfra, jedna wielka litera). Jeśli hasło nie spełnia wymagań, zgłaszany jest wyjątek `NieprawidloweHasloException`. Wyjątek jest przechwytywany i wyświetlany jest odpowiedni komunikat.

Zadanie 3. Obsługa wyjątków w aplikacji bankowej

Napisz prostą aplikację bankową, która pozwala na wpłacanie i wypłacanie środków z konta. Użyj instrukcji `throw`, aby zgłosić wyjątek, jeśli użytkownik spróbuje wypłacić więcej środków niż jest dostępnych na koncie. Dodaj również obsługę wyjątków związanych z nieprawidłowym formatem danych wejściowych.

Wskazówka:

- Użyj instrukcji `throw` w przypadku próby wypłacenia większej kwoty niż stan konta.
- Użyj bloków `catch` do obsługi różnych typów wyjątków.

Program symuluje prostą aplikację bankową, która pozwala na wpłacanie i wypłacanie środków. W przypadku próby wypłacenia większej kwoty niż stan konta, zgłaszany jest wyjątek `NiewystarczajaceSrodkiException`. Program obsługuje również wyjątki związane z nieprawidłowym formatem liczby (`FormatException`), nieprawidłową kwotą (`ArgumentOutOfRangeException`) oraz nieprawidłową operacją (`InvalidOperationException`).

Kod Zagłady

Rozdział 1. Wyjątek, który zmienił wszystko

Biuro było ciche, jak zawsze o tej porze. Tylko delikatny szum klimatyzacji przerywał ciszę. Marek wpatrywał się w ekran swojego laptopa, przesuając palcami po klawiaturze. Linijki kodu migotały przed jego oczami, ale dziś nie był w stanie się skupić. Coś było nie tak. Coś, czego nie mógł zignorować.

— Hej, Marku — usłyszał za sobą głos Kasi. — Spóźniasz się na spotkanie. Zespół czeka.

— Zaraz będę — mruknął, nie odrywając wzroku od ekranu. — Muszę tylko coś sprawdzić.

Kasia westchnęła i odeszła, a Marek wrócił do swojego kodu. To był zwykły program do zarządzania danymi klientów, ale coś w nim nie grało. Od kilku dni zauważał dziwne błędy w logach. Błędy, które nie powinny się pojawiać. Błędy, które wyglądały jak... celowe działanie.

— Nie może być — szepnął do siebie, przewijając kod w dół.

Nagle jego wzrok zatrzymał się na fragmencie, który wyglądał niewinnie, ale coś w nim było nie tak. To była prosta instrukcja throw:

throw new NieoczekiwanyBładException("Krytyczny błąd systemu. Proszę skontaktować się z administratorem.");

Na pierwszy rzut oka nic dziwnego. Programiści często używali throw, aby zgłaszać błędy. Ale ten wyjątek nie pochodził z ich biblioteki. To była zupełnie nowa klasa, której Marek nigdy wcześniej nie widział. Ktoś dodał ją do kodu bez jego wiedzy.

— Co tu się dzieje? — mruknął, otwierając definicję klasy NieoczekiwanyBładException.

Kod wyglądał niewinnie, ale Marek wiedział, że w programowaniu nic nie jest takie, jak się wydaje. Każda linijka mogła kryć pułapkę. Każdy wyjątek mógł być narzędziem w czyichś rękach.

— Kto to napisał? — zapytał sam siebie, przeszukując historię commitów.

Nie znalazł niczego. Ktoś usunął ślady. Ktoś, kto wiedział, co robi.

Nagle jego telefon zadzwonił. Nieznany numer.

— Tak? — odezwał się ostrożnie.

— Marek, słuchaj uważnie — usłyszał zmieniony głos. — To, co znalazłeś, to tylko wierzchołek góry lodowej. Nie pytaj, kim jestem. Po prostu uważaj. Ten kod to więcej niż błąd. To broń.

— Co? O czym ty mówisz? — zapytał Marek, ale po drugiej stronie rozległ się tylko krótki sygnał.

Połączenie zostało przerwane.

Serce Marka zaczęło bić szybciej. Spojrzał na ekran, na tę niewinną instrukcję throw. Coś w niej było nie tak. Coś, co mogło zmienić wszystko.

— Muszę to zgłębić — pomyślał, otwierając nowy plik. — Ale jak? I komu mogę zaufać?

Nagle zdał sobie sprawę, że ten kod to nie tylko program. To była zagadka, a on właśnie stał się jej częścią. I nie miał pojęcia, jak daleko sięgały konsekwencje.

Czy Marek odkryje prawdę? Kto stoi za tajemniczym kodem? I jakie sekrety kryje ta niewinna instrukcja throw? Tego dowiemy się w kolejnych rozdziałach...